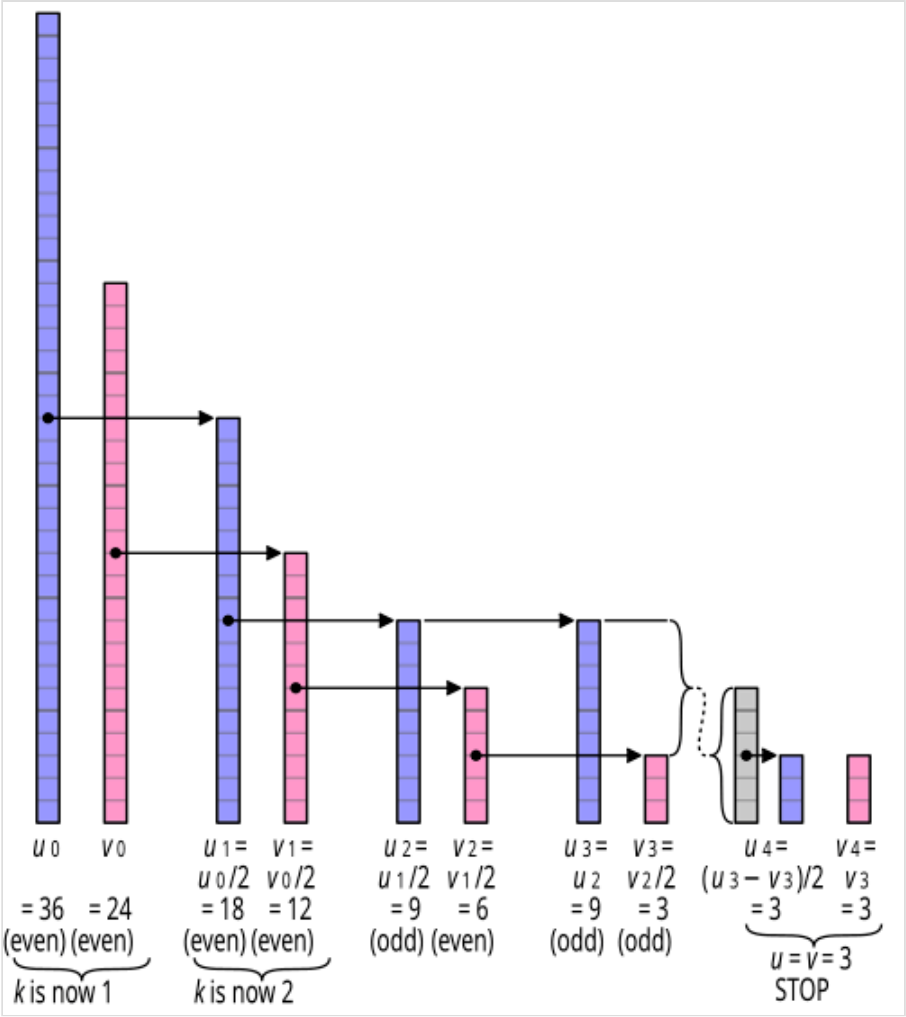
Binary GCD algorithm

The **binary GCD algorithm**, also known as **Stein's algorithm** or the **binary Euclidean algorithm**,[1][2] is an algorithm that computes the greatest common divisor (GCD) of two nonnegative integers. Stein's algorithm uses simpler arithmetic operations than the conventional Euclidean algorithm; it replaces division with arithmetic shifts, comparisons, and subtraction.

Although the algorithm in its contemporary form was first published by the physicist and programmer Josef Stein in 1967,[3] it was known by the 2nd century BCE, in ancient China.[4]



Visualisation of using the binary GCD algorithm to find the greatest common divisor (GCD) of 36 and 24. Thus, the GCD is $2^2 \times 3 = 12$.

Algorithm

The algorithm finds the GCD of two nonnegative numbers u and v by repeatedly applying these identities:

1. $\gcd(u, 0) = u$: everything divides zero, and u is the largest number that divides u.
2. $\gcd(2u, 2v) = 2 \cdot \gcd(u, v)$: 2 is a common divisor.
3. $\gcd(u, 2v) = \gcd(u, v)$ if u is odd: 2 is then not a common divisor.
4. $\gcd(u, v) = \gcd(u, v - u)$ if u, v odd and $u \leq v$.

As GCD is commutative ($\gcd(u, v) = \gcd(v, u)$), those identities still apply if the operands are swapped: $\gcd(0, v) = v$, $\gcd(2u, v) = \gcd(u, v)$ if v is odd, etc.

Implementation

While the above description of the algorithm is mathematically correct, the binary GCD algorithm performs more loop iterations that the Euclidean one, so it only offers a performance advantage if those iterations are substantially faster. Performant software implementations typically differ from it in a few notable ways:

- eschewing repeated trial division by 2 in favour of the count trailing zeros primitive and a single bit shift, which is functionally equivalent to repeatedly applying identity 3, but much faster;
- expressing the algorithm iteratively rather than recursively: the resulting implementation can be laid out to avoid repeated work, invoking identity 2 at the start and maintaining as invariant that both numbers are odd upon entering the loop, which only needs to implement identities 3 and 4; and
- making the loop's body branch-free except for its exit condition `v == 0`: unpredictable branches can have a large, negative impact on performance.[5][6]

The following is an implementation of the algorithm in C. After the shift at the top of the main loop, the variables `u` and `v` hold the values $(u\text{-}1)/2$ and $(v\text{-}1)/2$. This elimination of the least-significant bit leaves room for the signed difference `u-v` to be computed on the next line without overflow.

```
// Count trailing zeros.  This is a pedagogical example only;
// for efficiency use __builtin_ctzll() or similar.
static int ctz(uint64_t n)  {
    int s = 0, t;
    while ((t = n & 15) == 0) { n >>= 4; s += 4; }
    return s + ((0x12131210 >> 2*t) & 3);
}

uint64_t gcd(uint64_t u, uint64_t v)  {
    uint64_t uv = u | v;
    if (!u || !v) return uv;    // Identity #1

    u >>= ctz(u) + 1;
    int tz = ctz(v);
    for (;;) {
        v >>= tz + 1;           // Identity #3
        v -= u;                 // Identity #4
        if (v == 0) break;
        tz = ctz(v);            // ctz(v) == ctz(-v), so do this early
        // if ((int64_t)v < 0) { u += v; v = -v; }
        uint64_t mask = (int64_t)v >> 63;
        u += v & mask;          // Branch-free conditional add
        v ^= mask;              // Branch-free conditional negate
    }
    return (2*u+1) << ctz(uv);  // Identity #2
}
```

The following is an implementation of the algorithm in Rust exemplifying those differences, adapted from *uutils* (https://github.com/uutils/coreutils/blob/31087e821dc81dfbe4ed97980ae1faa19cbb3250/src/uu/factor/src/numeric/gcd.rs). The conditional exchange of u and v (ensuring $u \leq v$) compiles down to conditional moves;[7]:

```
pub fn gcd(mut u: u64, mut v: u64) -> u64 {
    // Base cases: gcd(n, 0) = gcd(0, n) = n
    if u == 0 {
```

```
        return v;
    } else if v == 0 {
        return u;
    }

    // Using identities 2 and 3:
    // gcd(2ⁱ u, 2ʲ v) = 2ᵏ gcd(u, v) with u, v odd and k = min(i, j)
    // 2ᵏ is the greatest power of two that divides both 2ⁱ u and 2ʲ v
    let i = u.trailing_zeros();  u >>= i;
    let j = v.trailing_zeros();  v >>= j;
    let k = i.min(j);

    loop {
        // u and v are odd at the start of the loop
        debug_assert!(u % 2 == 1, "u = {} should be odd", u);
        debug_assert!(v % 2 == 1, "v = {} should be odd", v);

        // Swap if necessary so u ≤ v
        if u > v {
            (u, v) = (v, u);
        }

        // Identity 4: gcd(u, v) = gcd(u, v-u) as u ≤ v and u, v are both odd
        v -= u;
        // v is now even

        if v == 0 {
            // Identity 1: gcd(u, 0) = u
            // The shift by k is necessary to add back the 2ᵏ factor that was removed before the loop
            return u << k;
        }

        // Identity 3: gcd(u, 2ʲ v) = gcd(u, v) as u is odd
        v >>= v.trailing_zeros();
    }
}
```

Note: The implementation above accepts *unsigned* (non-negative) integers; given that $\gcd(u, v) = \gcd(\pm u, \pm v)$, the signed case can be handled as follows:

```
/// Computes the GCD of two signed 64-bit integers
/// The result is unsigned and not always representable as i64: gcd(i64::MIN, i64::MIN) == 1 << 63
pub fn signed_gcd(u: i64, v: i64) -> u64 {
    gcd(u.unsigned_abs(), v.unsigned_abs())
}
```

Complexity

Asymptotically, the algorithm requires $O(n)$ steps, where n is the number of bits in the larger of the two numbers, as every two steps reduce at least one of the operands by at least a factor of 2. Each step involves only a few arithmetic operations ($O(1)$ with a small constant); when working with word-sized numbers, each arithmetic operation translates to a single machine operation, so the number of machine operations is on the order of n, i.e. $\log_2(\max(u, v))$.

For arbitrarily large numbers, the asymptotic complexity of this algorithm is $O(n^2)$,[8] as each arithmetic operation (subtract and shift) involves a linear number of machine operations (one per word in the numbers' binary representation). If the numbers can be represented in the machine's memory, *i.e.* each number's *size* can be represented by a single machine word, this bound is reduced to:

$$O\left(\frac{n^2}{\log_2 n}\right)$$

This is the same as for the Euclidean algorithm, though a more precise analysis by Akhavi and Vallée proved that binary GCD uses about 60% fewer bit operations.[9]

Extensions

The binary GCD algorithm can be extended in several ways, either to output additional information, deal with arbitrarily large integers more efficiently, or to compute GCDs in domains other than the integers.

The *extended binary GCD* algorithm, analogous to the extended Euclidean algorithm, fits in the first kind of extension, as it provides the Bézout coefficients in addition to the GCD: integers a and b such that $a \cdot u + b \cdot v = \gcd(u, v)$.[10][11][12]

In the case of large integers, the best asymptotic complexity is $O(M(n) \log n)$, with $M(n)$ the cost of n-bit multiplication; this is near-linear and vastly smaller than the binary GCD algorithm's $O(n^2)$, though concrete implementations only outperform older algorithms for numbers larger than about 64 kilobits (*i.e.* greater than 8×10^{19265}). This is achieved by extending the binary GCD algorithm using ideas from the Schönhage–Strassen algorithm for fast integer multiplication.[13]

The binary GCD algorithm has also been extended to domains other than natural numbers, such as Gaussian integers,[14] Eisenstein integers,[15] quadratic rings,[16][17] and integer rings of number fields.[18]

Historical description

An algorithm for computing the GCD of two numbers was known in ancient China, under the Han dynasty, as a method to reduce fractions:

> If possible halve it; otherwise, take the denominator and the numerator, subtract the lesser from the greater, and do that alternately to make them the same. Reduce by the same number.

—Fangtian – Land surveying, *The Nine Chapters on the Mathematical Art*

The phrase "if possible halve it" is ambiguous,[4]

- if this applies when *either* of the numbers become even, the algorithm is the binary GCD algorithm;
- if this only applies when *both* numbers are even, the algorithm is similar to the Euclidean algorithm.

See also

Computer programming portal

- Euclidean algorithm
- Extended Euclidean algorithm
- Least common multiple

References

1. Brent, Richard P. (13–15 September 1999). *Twenty years' analysis of the Binary Euclidean Algorithm* (http://wwwmaths.anu.edu.au/~brent/pub/pub183.html). 1999 Oxford-Microsoft Symposium in honour of Professor Sir Antony Hoare. Oxford.
2. Brent, Richard P. (November 1999). *Further analysis of the Binary Euclidean algorithm* (https://www.cs.ox.ac.uk/techreports/oucl/tr-7-99.html) (Technical report). Oxford University Computing Laboratory. arXiv:1303.2772 (https://arxiv.org/abs/1303.2772). PRG TR-7-99.
3. Stein, J. (February 1967), "Computational problems associated with Racah algebra", *Journal of Computational Physics*, **1** (3): 397–405, Bibcode:1967JCoPh...1..397S (https://ui.adsabs.harvard.edu/abs/1967JCoPh...1..397S), doi:10.1016/0021-9991(67)90047-2 (https://doi.org/10.1016%2F0021-9991%2867%2990047-2), ISSN 0021-9991 (https://search.worldcat.org/issn/0021-9991)
4. Knuth, Donald (1998), *Seminumerical Algorithms*, The Art of Computer Programming, vol. 2 (3rd ed.), Addison-Wesley, ISBN 978-0-201-89684-8
5. Kapoor, Rajiv (21 February 2009). "Avoiding the Cost of Branch Misprediction" (https://software.intel.com/content/www/us/en/develop/articles/avoiding-the-cost-of-branch-misprediction.html). *Intel Developer Zone*.
6. Lemire, Daniel (15 October 2019). "Mispredicted branches can multiply your running times" (https://lemire.me/blog/2019/10/15/mispredicted-branches-can-multiply-your-running-times/).
7. Godbolt, Matt. "Compiler Explorer" (https://rust.godbolt.org/z/56jva3KPn). Retrieved 4 February 2024.
8. "GNU MP 6.1.2: Binary GCD" (http://gmplib.org/manual/Binary-GCD.html).
9. Akhavi, Ali; Vallée, Brigitte (2000), "Average Bit-Complexity of Euclidean Algorithms" (https://vallee.users.greyc.fr/Publications/icalp8-2000.ps), *Proceedings ICALP'00, Lecture Notes Computer Science 1853*: 373–387, CiteSeerX 10.1.1.42.7616 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.42.7616)
10. Knuth 1998, p. 646, answer to exercise 39 of section 4.5.2
11. Menezes, Alfred J.; van Oorschot, Paul C.; Vanstone, Scott A. (October 1996). "§14.4 Greatest Common Divisor Algorithms" (http://cacr.uwaterloo.ca/hac/about/chap14.pdf#page=17) (PDF). *Handbook of Applied Cryptography* (http://cacr.uwaterloo.ca/hac/). CRC Press. pp. 606–610. ISBN 0-8493-8523-7. Retrieved 9 September 2017.
12. Cohen, Henri (1993). "Chapter 1 : Fundamental Number-Theoretic Algorithms". *A Course In Computational Algebraic Number Theory* (https://books.google.com/books?id=hXGr-9l1DXcC). Graduate Texts in Mathematics. Vol. 138. Springer-Verlag. pp. 17–18. ISBN 0-387-55640-0.

13. Stehlé, Damien; Zimmermann, Paul (2004), "A binary recursive gcd algorithm" (http://hal.archives-ouvertes.fr/docs/00/07/15/33/PDF/RR-5050.pdf) (PDF), *Algorithmic number theory*, Lecture Notes in Comput. Sci., vol. 3076, Springer, Berlin, pp. 411–425, CiteSeerX 10.1.1.107.8612 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.107.8612), doi:10.1007/978-3-540-24847-7_31 (https://doi.org/10.1007%2F978-3-540-24847-7_31), ISBN 978-3-540-22156-2, MR 2138011 (https://mathscinet.ams.org/mathscinet-getitem?mr=2138011), S2CID 3119374 (https://api.semanticscholar.org/CorpusID:3119374), INRIA Research Report RR-5050.
14. Weilert, André (July 2000). "(1+i)-ary GCD Computation in Z[i] as an Analogue to the Binary GCD Algorithm" (https://doi.org/10.1006%2Fjsco.2000.0422). *Journal of Symbolic Computation*. **30** (5): 605–617. doi:10.1006/jsco.2000.0422 (https://doi.org/10.1006%2Fjsco.2000.0422).
15. Damgård, Ivan Bjerre; Frandsen, Gudmund Skovbjerg (12–15 August 2003). *Efficient Algorithms for GCD and Cubic Residuosity in the Ring of Eisenstein Integers*. 14th International Symposium on the Fundamentals of Computation Theory. Malmö, Sweden. pp. 109–117. doi:10.1007/978-3-540-45077-1_11 (https://doi.org/10.1007%2F978-3-540-45077-1_11).
16. Agarwal, Saurabh; Frandsen, Gudmund Skovbjerg (13–18 June 2004). *Binary GCD Like Algorithms for Some Complex Quadratic Rings*. Algorithmic Number Theory Symposium. Burlington, VT, USA. pp. 57–71. doi:10.1007/978-3-540-24847-7_4 (https://doi.org/10.1007%2F978-3-540-24847-7_4).
17. Agarwal, Saurabh; Frandsen, Gudmund Skovbjerg (20–24 March 2006). *A New GCD Algorithm for Quadratic Number Rings with Unique Factorization*. 7th Latin American Symposium on Theoretical Informatics. Valdivia, Chile. pp. 30–42. doi:10.1007/11682462_8 (https://doi.org/10.1007%2F11682462_8).
18. Wikström, Douglas (11–15 July 2005). *On the I-Ary GCD-Algorithm in Rings of Integers*. Automata, Languages and Programming, 32nd International Colloquium. Lisbon, Portugal. pp. 1189–1201. doi:10.1007/11523468_96 (https://doi.org/10.1007%2F11523468_96).

Further reading

- Knuth, Donald (1998). "§4.5 Rational arithmetic". *Seminumerical Algorithms*. The Art of Computer Programming. Vol. 2 (3rd ed.). Addison-Wesley. pp. 330–417. ISBN 978-0-201-89684-8.

Covers the extended binary GCD, and a probabilistic analysis of the algorithm.

- Cohen, Henri (1993). "Chapter 1 : Fundamental Number-Theoretic Algorithms". *A Course In Computational Algebraic Number Theory* (https://books.google.com/books?id=hXGr-9l1DXcC). Graduate Texts in Mathematics. Vol. 138. Springer-Verlag. pp. 12–24. ISBN 0-387-55640-0.

Covers a variety of topics, including the extended binary GCD algorithm which outputs Bézout coefficients, efficient handling of multi-precision integers using a variant of Lehmer's GCD algorithm, and the relationship between GCD and continued fraction expansions of real numbers.

- Vallée, Brigitte (September–October 1998). "Dynamics of the Binary Euclidean Algorithm: Functional Analysis and Operators" (https://web.archive.org/web/20110513012901/http://users.info.unicaen.fr/~brigitte/Publications/bin-gcd.ps). *Algorithmica*. **22** (4): 660–685. doi:10.1007/PL00009246 (https://doi.org/10.1007%2FPL00009246). S2CID 27441335 (https://api.semanticscholar.org/CorpusID:27441335). Archived from the original (https://users.info.unicaen.fr/~brigitte/Publications/bin-gcd.ps) (PS) on 13 May 2011.

An analysis of the algorithm in the average case, through the lens of functional analysis: the algorithms' main parameters are cast as a dynamical system, and their average value is related to the invariant measure of the system's transfer operator.

External links

- NIST Dictionary of Algorithms and Data Structures: binary GCD algorithm (https://xlinux.nist.gov/dads/HTML/binaryGCD.html)
- Cut-the-Knot: Binary Euclid's Algorithm (http://www.cut-the-knot.org/blue/binary.shtml) at cut-the-knot
- *Analysis of the Binary Euclidean Algorithm* (http://wwwmaths.anu.edu.au/~brent/pub/pub037.html) (1976), a paper by Richard P. Brent, including a variant using left shifts

Retrieved from "https://en.wikipedia.org/w/index.php?title=Binary_GCD_algorithm&oldid=1343331567"